

程式設計 Structures & Class

張賢宗

長庚大學人工智慧學系

Learning Objectives

❖ Structures

- Structure types
- Structures as function arguments
- Initializing structures

Structures

- ❖ 2nd aggregate data type: struct
- ❖ Recall: aggregate meaning "grouping"
 - Recall array: collection of values of same type
 - Structure: collection of values of different types
- ❖ Treated as a single item, like arrays
- ❖ Major difference: Must first "define" struct
 - Prior to declaring any variables

Structure Types

- ❖ Define struct globally (typically)
- ❖ No memory is allocated
 - Just a "placeholder" for what our struct will "look like"

- ❖ Definition:

```
struct CDAccountV1  ← Name of new struct "type"
{
    double balance;    ← member names
    double interestRate;
    int term;
};
```

Declare Structure Variable

- ❖ With structure type defined, now declare variables of this new type:
CDAccountV1 account;
 - Just like declaring simple types
 - Variable *account* now of type CDAccountV1
 - It contains "member values"
 - Each of the struct "parts"

Accessing Structure Members

❖ Dot Operator to access members

- `account.balance`
- `account.interestRate`
- `account.term`

❖ Called "member variables"

- The "parts" of the structure variable
- Different structs can have same name member variables
 - No conflicts

Structure Example: Display 6.1 A Structure Definition (1 of 3)

Display 6.1 A Structure Definition

```
1 //Program to demonstrate the CDAccountV1 structure type.
2 #include <iostream>
3 using namespace std;

4 //Structure for a bank certificate of deposit:
5 struct CDAccountV1
6 {
7     double balance;
8     double interestRate;
9     int term;//months until maturity
10 };

11 void getData(CDAccountV1& theAccount);
12 //Postcondition: theAccount.balance, theAccount.interestRate, and
13 //theAccount.term have been given values that the user entered at the keyboar
```

An improved version of this structure will be given later in this chapter.

Structure Example: Display 6.1 A Structure Definition (2 of 3)

```
14  int main()  
15  {  
16      CDAccountV1 account;  
17      getData(account);  
  
18      double rateFraction, interest;  
19      rateFraction = account.interestRate/100.0;  
20      interest = account.balance*(rateFraction*(account.term/12.0));  
21      account.balance = account.balance + interest;  
  
22      cout.setf(ios::fixed);  
23      cout.setf(ios::showpoint);  
24      cout.precision(2);  
25      cout << "When your CD matures in "  
26           << account.term << " months,\n"  
27           << "it will have a balance of $"  
28           << account.balance << endl;  
  
29      return 0;  
30  }
```

(continued)

Structure Example: Display 6.1 A Structure Definition (3 of 3)

Display 6.1 A Structure Definition

```
31 //Uses iostream:
32 void getData(CDAccountV1& theAccount)
33 {
34     cout << "Enter account balance: $";
35     cin >> theAccount.balance;
36     cout << "Enter account interest rate: ";
37     cin >> theAccount.interestRate;
38     cout << "Enter the number of months until maturity: ";
39     cin >> theAccount.term;
40 }
```

SAMPLE DIALOGUE

Enter account balance: \$100.00
Enter account interest rate: 10.0
Enter the number of months until maturity: 6
When your CD matures in 6 months,
it will have a balance of \$105.00



widelab
Web Information and Data Engineering Laboratory

Structure Pitfall

❖ Semicolon after structure definition

- ; **MUST** exist:

```
struct WeatherData
```

```
{
```

```
    double temperature;
```

```
    double windVelocity;
```

```
}; ← REQUIRED semicolon!
```

- Required since you "can" declare structure variables in this location



widelab
Web Information and Data Engineering Laboratory

Structure Assignments

- ❖ Given structure named CropYield
- ❖ Declare two structure variables:
CropYield apples, oranges;
 - Both are variables of "struct type CropYield"
 - Simple assignments are legal:
apples = oranges;
 - Simply copies each member variable from apples into member variables from oranges

說明

❖ Int a,b;

- `a = b;` //表示a等於b

❖ Employee a,b;

- `a=b;`

// 表示a裡面所有member等於b的member

- `a.firstman = b.firstman`
- `a.age = b.age`



Self-test

❖ 設計一個staff 資料結構,裡面包括

- double WorkingHour,
- int salaryPerHour, age ;
- char department;

❖ 宣告兩個staff 變數 A, B,Max

Staff	WorkingHour	SalaryPerHour	age	department
A	40	1000	40	H
B	80	780	30	C

❖ 找出薪水最高的員工,將結果存於Max後,顯示各member內容

Initializing Structures

❖ Can initialize at declaration

- Example:

```
struct Date
```

```
{
```

```
    int month;
```

```
    int day;
```

```
    int year;
```

```
};
```

```
    Date dueDate = {12, 31, 2003};
```

- Declaration provides initial data to all three member variables

Structures as Function Arguments

- ❖ Passed like any simple data type
 - Pass-by-value
 - Pass-by-reference
 - Or combination
- ❖ Can also be returned by function
 - Return-type is structure type
 - Return statement in function definition sends structure variable back to caller

Example

❖ struct Date

```
{  
    int month;  
    int day;  
    int year;
```

```
};
```

```
Date Getdate();
```

❖ Date Getdate()

```
{  
    Date temp;  
    cout << "Please enter month" ;  
    cin>>temp.month;  
    cin>>temp.day;  
    cin>>temp.year;  
    return(temp);  
}
```

• Main ()

```
{  
    Date Newdate;  
    Newdate = Getdate ();  
    cout<<"mm-dd-yy :" <<endl;  
    cout<<Newdate.month;  
    cout<<Newdate.day;  
    cout<<Newdate.year;  
}
```

Example

- Main ()

```
struct Date
{
    int month;
    int day;
    int year;
};
Date Getdate();
Void PrintDate(Date temp);
Date Getdate()
{
    Date temp;
    cout << "Please enter month" ;
    cin>>temp.month;
    cin>>temp.day;
    cin>>temp.year;
    return(temp);
}
```

```
{
    Date Newdate;
    Newdate = Getdate ();
    PrintDate( Newdate);
}
void PrintDate(Date temp)
{
    cout<<"mm-dd-yy :" <<endl;
    cout<<temp.month;
    cout<<temp.day;
    cout<<temp.year;
}
```



widelab
Web Information and Data Engineering Laboratory

Hierarchical structures

❖ struct Date

```
{  
    int month;  
    int day;  
    int year;  
};
```

❖ struct PersonInfo

```
{  
    double height,weight;  
    int age;  
    Date birthday;  
};
```

• main()

```
{  
    PersonInfo A;  
    cin>>A.birthday.month;  
};
```

Self test

- struct Date

```
{  
    int month;  
    int day;  
    int year;  
};
```

- struct PersonInfo

```
{  
    double height,weight;  
    int age;  
    Date birthday;  
};
```

- ❖ 建立Date與PersonInfo兩個結構
- ❖ 建立 void Getdate(Date& thedate),完成生日的輸入
- ❖ 建立 PersonInfo GetPerson(),完成個人資料的輸入
- ❖ 利用陣列產生5個人的相關基本資料
- ❖ 身高最高的陣列索引值是?

Self Test

- ❖ 建立學生的資料結構,內容包括
 - 整數 學號,計概,微積分,普物
 - 浮點數 平均
- ❖ 利用陣列產生50個學生,並用亂數填入各科值(0 - 100)
- ❖ 完成下列功能
 - 計算平均
 - 利用selection sort排序,顯示運算時間
 - 利用bubble sort排序,顯示運算時間

Summary

❖ Structure is collection of different types

Class Concepts

Learning Objectives

❖ Classes

- Defining, member functions
- Public and private members
- Accessor and mutator functions
- Structures vs. classes

Classes

- ❖ Similar to structures
 - Adds member FUNCTIONS
 - Not just member data
- ❖ Integral to object-oriented programming(物件導向程式的核心技術)
 - Focus on objects
 - Object: Contains data and operations
 - In C++, variables of class type are objects

Class Definitions

- ❖ Defined similar to structures

- ❖ Example:

```
class DayOfYear ← name of new class type
{
public:
    void output(); ← member function!
    int month; ← member variable!
    int day;
};
```

- ❖ Notice only member function' s prototype
 - Function' s implementation is elsewhere

Declaring Objects

- ❖ Declared same as all variables
 - Predefined types, structure types
- ❖ Example:
DayOfYear today, birthday;
 - Declares two objects of class type DayOfYear
- ❖ Objects include:
 - Data
 - Members month, day
 - Operations (member functions)
 - output()

Class Member Access

- ❖ Members accessed same as structures
- ❖ Example:
 - today.month
 - today.day
 - And to access member function:
today.output(); ← Invokes member function

Class Member Functions

- ❖ Must define or "implement" class member functions
- ❖ Like other function definitions
 - Can be after main() definition
 - Must specify class:
void DayOfYear::output()
{...}
 - :: is scope resolution operator
 - class 用 ::, similar to that the . ' is used with object
 - Instructs compiler "what class" member is from
 - Item before :: called type qualifier (類型限定)

Dot and Scope Resolution Operator

- ❖ Used to specify "of what thing" they are members
- ❖ Dot operator:
 - Specifies member of particular object
- ❖ Scope resolution operator:
 - Specifies what class the function definition comes from

Class Member Functions Definition

- ❖ Notice output() member function's definition (in next example)
- ❖ Refers to member data of class
 - No qualifiers
- ❖ Function used for all objects of the class
 - Will refer to "that object's" data when invoked
 - Example:
today.output();
 - Displays "today" object's data

Complete Class Example: Display 6.3 Class With a Member Function (1 of 4)

Display 6.3 Class with a Member Function

```
1 //Program to demonstrate a very simple example of a class.
2 //A better version of the class DayOfYear will be given in Display 6.4.
3 #include <iostream>
4 using namespace std;

5 class DayOfYear
6 {
7 public:
8     void output( );
9     int month;
10    int day;
11 };

12 int main( )
13 {
14     DayOfYear today, birthday;
15     cout << "Enter today's date:\n";
16     cout << "Enter month as a number: ";
17     cin >> today.month;
18     cout << "Enter the day of the month: ";
19     cin >> today.day;
20     cout << "Enter your birthday:\n";
21     cout << "Enter month as a number: ";
22     cin >> birthday.month;
23     cout << "Enter the day of the month: ";
24     cin >> birthday.day;
```

Normally, member variables are private and not public, as in this example. This is discussed a bit later in this chapter.

Member function declaration

(continued)

Complete Class Example: Display 6.3 Class With a Member Function (2 of 4)

Display 6.3 Class with a Member Function

```

25     cout << "Today's date is ";
26     today.output( );
27     cout << endl;
28     cout << "Your birthday is ";
29     birthday.output( );
30     cout << endl;

31     if (today.month == birthday.month && today.day == birthday.day)
32         cout << "Happy Birthday!\n";
33     else
34         cout << "Happy Unbirthday!\n";
35     return 0;
36 }
37 //Uses iostream:
38 void DayOfYear::output( )
39 {
40     switch (month)
41     {
42         case 1:
43             cout << "January "; break;
44         case 2:
45             cout << "February "; break;
46         case 3:
47             cout << "March "; break;
48         case 4:
49             cout << "April "; break;

```

Calls to the member function output

Member function definition

today.output(),此時相當於傳入
today.month

Complete Class Example: Display 6.3 Class With a Member Function (3 of 4)

```
50         case 5:
51             cout << "May "; break;
52         case 6:
53             cout << "June "; break;
54         case 7:
55             cout << "July "; break;
56         case 8:
57             cout << "August "; break;
58         case 9:
59             cout << "September "; break;
60         case 10:
61             cout << "October "; break;
62         case 11:
63             cout << "November "; break;
64         case 12:
65             cout << "December "; break;
66         default:
67             cout << "Error in DayOfYear::output. Contact software vendor.";
68     }
69
70     cout << day;
71 }
```

Complete Class Example: Display 6.3 Class With a Member Function (4 of 4)

Display 6.3 Class with a Member Function

SAMPLE DIALOGUE

Enter today's date:
Enter month as a number: 10
Enter the day of the month: 15
Enter your birthday:
Enter month as a number: 2
Enter the day of the month: 21
Today's date is October 15
Your birthday is February 21
Happy Unbirthday!

Self test exercise

❖ 設計一個class Dayofmonth並完成相關函式

❖ class Dayofmonth

```
{
```

```
    public:
```

```
        void input(); //輸入變數month的值
```

```
        void output(); //輸出變數month的值
```

```
        int month;
```

```
};
```



widelab
Web Information and Data Engineering Laboratory

Public and Private Members

- ❖ Data in class almost always designated private in definition!
 - Upholds principles of OOP
 - Hide data from user
 - Allow manipulation only via operations
 - Which are member functions
- ❖ Public items (usually member functions) are "user-accessible"



Public and Private Example

- ❖ Modify previous example:

```
class DayOfYear
{
public:
    void input();
    void output();
private:
    int month;
    int day;
};
```

- ❖ Data now private

- ❖ Objects have no direct access

Public and Private Example 2

- ❖ Given previous example
- ❖ Declare object:
DayOfYear today;
- ❖ Object *today* can ONLY access public members
 - `cin >> today.month; // NOT ALLOWED!`
 - `cout << today.day; // NOT ALLOWED!`
 - Must instead call public operations:
 - `today.input();`
 - `today.output();`

Public and Private Style

- ❖ Can mix & match public & private
- ❖ More typically place public first
 - Allows easy viewing of portions that can be USED by programmers using the class
 - Private data is "hidden", so irrelevant to users
- ❖ Outside of class definition, cannot change (or even access) private data



widelab
Web Information and Data Engineering Laboratory

Accessor and Mutator Functions

- ❖ Object needs to "do something" with its data
- ❖ Call accessor (getter) member functions
 - Allow object to read data
 - Also called "get member functions"
 - Simple retrieval of member data
- ❖ Mutator (setter) member functions
 - Allow object to change data
 - Manipulated based on application
- ❖ Make sure the member data are "correct" stored.

Example

❖ Class DayofYear

```
{  
    public:  
        void set (int newMonth); // mutator  
        int getMonth();  
    private:  
        int month;  
};
```

```
❖ Void DayofYear ::set(int newMonth)
{
    if ((newMonth>=1) && newMonth<12))
        month = newMonth;
    else
    {
        cout<< " Illegal month value" ;
        exit(1);
    }
}
```

Example

```
• Int DayofYear::getMonth()
{
    return (month);
}
int main()
{
    DayofYear birthday;
    birthday.set(12);
    cout << birthday.getMonth();
}
```

Self-test

❖ class A

```
{  
    public:  
        void setP (double NP);  
        double getP();  
    private:  
        double P;  
}
```

- ❖ 完成左邊class的實作
- ❖ 利用setP()將p值成功指定，如果大於10000，則顯示輸入值超出範圍
- ❖ 利用getP()顯示設定結果

Separate Interface and Implementation

- ❖ User of class need not see details of how class is implemented
 - Principle of OOP → encapsulation
- ❖ User only needs "rules"
 - Called "interface" for the class
 - In C++ → public member functions and associated comments
- ❖ Implementation of class hidden
 - Member function definitions elsewhere
 - User need not see them

Structures versus Classes

❖ Structures

- Typically all members public
- No member functions

❖ Classes

- Typically all data members private
- Interface member functions public

❖ Technically, same

- Perceptually, very different mechanisms
- Structure 適合只需要組織相關資料
- Class 適合需要封裝、繼承甚至多型的特性，且具有功能性。

Thinking Objects

- ❖ Focus for programming changes
 - Before → algorithms(procedure or function) center stage
 - OOP → data(Object/class) is focus
- ❖ Algorithms still exist
 - They simply focus on their data
 - Are "made" to "fit" the data
- ❖ Designing software solution
 - Define variety of objects and how they interact
- ❖ OOP 更適合複雜或大型的資料程式。Procedural 寫起來比較簡便，一般來說效能也較好。

Constructors

- ❖ Initialization of objects
 - Initialize some or all member variables
 - Other actions possible as well
- ❖ A special kind of member function
 - Automatically called when object declared
- ❖ Very useful tool
 - Key principle of OOP

Constructor Definitions

- ❖ Constructors defined like any member function
 - Except:
 1. Must have same name as class
 2. Cannot return a value; not even void! -> **No return type.**
 3. Member data should always available (and correct) when one object is borne.



Constructor Definition Example

- Class definition with constructor:
 - ```
class DayOfYear
{
public:
 DayOfYear(int monthValue, int dayValue);
 //Constructor initializes month & day
 void input();
 void output();

private:
 int month;
 int day;
}
```

# Constructor Code

- ❖ Constructor definition is like all other member functions:  

```
DayOfYear::DayOfYear(int monthValue, int dayValue)
{
 month = monthValue;
 day = dayValue;
}
```
- ❖ Note same name around ::
  - Clearly identifies a constructor
- ❖ Note no return type
  - Just as in class definition

# Constructor Notes

- ❖ Notice name of constructor: DayOfYear
  - Same name as class itself!
- ❖ Constructor declaration has no return-type
  - Not even void!
- ❖ Constructor in public section
  - It's called when objects are declared
  - If private, could never declare objects!

# Calling Constructors

- ❖ Declare objects:  
DayOfYear date1(7, 4),  
date2(5, 5);
- ❖ Objects are created here
  - Constructor is called
  - Values in parens passed as arguments to constructor
  - Member variables month, day initialized:  
date1.month → 7 date2.month → 5  
date1.day → 4 date2.day → 5

# Constructor Equivalency

## ❖ Consider:

- DayOfYear date1, date2  
date1.DayOfYear(7, 4); // ILLEGAL!  
date2.DayOfYear(5, 5); // ILLEGAL!

## ❖ Seemingly OK...

- CANNOT call constructors like other member functions!

# Alternative Definition

- ❖ Previous definition equivalent to:

```
DayOfYear::DayOfYear(int monthValue,
 int dayValue)
 : month(monthValue), day(dayValue) ←
 {...}
```

- ❖ Third line called "Initialization Section"
- ❖ Body left empty
- ❖ Preferable definition version



widelab  
Web Information and Data Engineering Laboratory

## Constructor Additional Purpose

- ❖ Not just initialize data
- ❖ Body doesn't have to be empty
  - In initializer version
- ❖ Validate the data!
  - Ensure only appropriate data is assigned to class private member variables
  - Powerful OOP principle

# Overloaded Constructors

- ❖ Can overload constructors just like other functions
- ❖ Recall: a signature consists of:
  - Name of function
  - Parameter list
- ❖ Provide constructors for all possible argument-list

## Class with Constructors Example: Display 7.1 Class with Constructors (1 of 3)

### Display 7.1 Class with Constructors

```
1 #include <iostream>
2 #include <cstdlib> //for exit
3 using namespace std;

4 class DayOfYear
5 {
6 public:
7 DayOfYear(int monthValue, int dayValue);
8 //Initializes the month and day to arguments.

9 DayOfYear(int monthValue);
10 //Initializes the date to the first of the given month.

11 DayOfYear(); ← default constructor
12 //Initializes the date to January 1.

13 void input();
14 void output();
15 int getMonthNumber();
16 //Returns 1 for January, 2 for February, etc.
```

## Class with Constructors Example: Display 7.1 Class with Constructors (2 of 3)

```

17 int getDay();
18 private:
19 int month;
20 int day;
21 void testDate();
22 };

```

```

23 int main()
24 {
25 DayOfYear date1(2, 21), date2(5), date3;
26 cout << "Initialized dates:\n";
27 date1.output(); cout << endl;
28 date2.output(); cout << endl;
29 date3.output(); cout << endl;

```

```

30 date1 = DayOfYear(10, 31);
31 cout << "date1 reset to the following:\n";
32 date1.output(); cout << endl;
33 return 0;
34 }

```

```

35
36 DayOfYear::DayOfYear(int monthValue, int dayValue)
37 : month(monthValue), day(dayValue)
38 {
39 testDate();
40 }

```

*This causes a call to the default constructor. Notice that there are no parentheses.*

*an explicit call to the constructor  
DayOfYear::DayOfYear*

## Class with Constructors Example: Display 7.1 Class with Constructors (3 of 3)

**Display 7.1 Class with Constructors**

```

41 DayOfYear::DayOfYear(int monthValue) : month(monthValue), day(1)
42 {
43 testDate();
44 }

45 DayOfYear::DayOfYear() : month(1), day(1)
46 { /*Body intentionally empty.*/}

47 //uses iostream and cstdlib:
48 void DayOfYear::testDate()
49 {
50 if ((month < 1) || (month > 12))
51 {
52 cout << "Illegal month value!\n";
53 exit(1);
54 }
55 if ((day < 1) || (day > 31))
56 {
57 cout << "Illegal day value!\n";
58 exit(1);
59 }
60 }

```

*<Definitions of the other member functions are the same as in Display 6.4.>*

### SAMPLE DIALOGUE

Initialized dates:  
February 21  
May 1  
January 1  
date1 reset to the following:  
October 31



## Constructor with No Arguments

- ❖ Can be confusing
- ❖ Standard functions with no arguments:
  - Called with syntax: `callMyFunction();`
    - Including empty parentheses
- ❖ Object declarations with no "initializers":
  - `DayOfYear date1; // This way!`
  - `DayOfYear date(); // NO!`
    - What is this really?
    - Compiler sees a function declaration/prototype!
    - Yes! Look closely!

# Explicit Constructor Calls

- ❖ Can also call constructor AGAIN
  - After object declared
    - Recall: constructor was automatically called then
  - Can call via object's name; standard member function call
- ❖ Convenient method of setting member variables
- ❖ Method quite different from standard member function call

## Explicit Constructor Call Example

❖ Such a call returns "anonymous object"

- Which can then be assigned

- In Action:

DayOfYear holiday(7, 4);

- Constructor called at object' s declaration
- Now to "re-initialize":  
holiday = DayOfYear(5, 5);
  - Explicit constructor call
  - Returns new "anonymous object"
  - Assigned back to current object

# Default Constructor

- ❖ Defined as: constructor that takes no arguments
- ❖ One should always be defined
- ❖ Auto-Generated?
  - Yes & No
  - If no constructors AT ALL are defined → Yes
  - If any constructors are defined → No
- ❖ If no default constructor:
  - Cannot declare: `MyClass myObject;`
    - With no initializers

## Class Type Member Variables

- ❖ Class member variables can be any type
  - Including objects of other classes!
  - Type of class relationship
    - Powerful OOP principle
- ❖ Need special notation for constructors
  - So they can call "back" to member object' s constructor

## Class Member Variable Example: Display 7.3 A Class Member Variable (1 of 5)

### Display 7.3 A Class Member Variable

---

```
1 #include <iostream>
2 #include<cstdlib>
3 using namespace std;

4 class DayOfYear
5 {
6 public:
7 DayOfYear(int monthValue, int dayValue);
8 DayOfYear(int monthValue);
9 DayOfYear();
10 void input();
11 void output();
12 int getMonthNumber();
13 int getDay();
14 private:
15 int month;
16 int day;
17 void testDate();
18 };
```

*The class DayOfYear is the same as in Display 7.1, but we have repeated all the details you need for this discussion.*

## Class Member Variable Example: Display 7.3 A Class Member Variable (2 of 5)

```

19 class Holiday
20 {
21 public:
22 Holiday();//Initializes to January 1 with no parking enforcement
23 Holiday(int month, int day, bool theEnforcement);
24 void output();
25 private:
26 DayOfYear date;
27 bool parkingEnforcement;//true if enforced
28 };

29 int main()
30 {
31 Holiday h(2, 14, true);
32 cout << "Testing the class Holiday.\n";
33 h.output();

34 return 0;
35 }

36
37 Holiday::Holiday() : date(1, 1), parkingEnforcement(false)
38 { /*Intentionally empty*/}

39 Holiday::Holiday(int month, int day, bool theEnforcement)
40 : date(month, day), parkingEnforcement(theEnforcement)
41 { /*Intentionally empty*/}

```

*member variable of a class type*

*Invocations of constructors from the class DayOfYear.*

(continued)

## Class Member Variable Example: Display 7.3 A Class Member Variable (3 of 5)

### Display 7.3 A Class Member Variable

---

```
42 void Holiday::output()
43 {
44 date.output();
45 cout << endl;
46 if (parkingEnforcement)
47 cout << "Parking laws will be enforced.\n";
48 else
49 cout << "Parking laws will not be enforced.\n";
50 }

51 DayOfYear::DayOfYear(int monthValue, int dayValue)
52 : month(monthValue), day(dayValue)
53 {
54 testDate();
55 }
```

## Class Member Variable Example: Display 7.3 A Class Member Variable (4 of 5)

```

56 //uses iostream and cstdlib:
57 void DayOfYear::testDate()
58 {
59 if ((month < 1) || (month > 12))
60 {
61 cout << "Illegal month value!\n";
62 exit(1);
63 }
64 if ((day < 1) || (day > 31))
65 {
66 cout << "Illegal day value!\n";
67 exit(1);
68 }
69 }
70
71 //Uses iostream:
72 void DayOfYear::output()
73 {
74 switch (month)
75 {
76 case 1:
77 cout << "January "; break;
78 case 2:
79 cout << "February "; break;
80 case 3:
81 cout << "March "; break;
82 .
83 .
84 .

```

*The omitted lines are in Display 6.3, but they are obvious enough that you should not have to look there.*

## Class Member Variable Example: Display 7.3 A Class Member Variable (5 of 5)

### Display 7.3 A Class Member Variable

---

```
82 case 11:
83 cout << "November "; break;
84 case 12:
85 cout << "December "; break;
86 default:
87 cout << "Error in DayOfYear::output. Contact software vendor.";
88 }

89 cout << day;
90 }
```

#### SAMPLE DIALOGUE

Testing the class Holiday.  
February 14  
Parking laws will be enforced.

---

# Self-test

```
❖ classYourClass
{
 public :
 YourClass(int newInfo, char moreInfo);
 YourClass();
 void output();

 private:
 int information;
 char moreInformation;
}
```

❖ Which of the following are legal?

- YourClass AAA(42, ' A' );
- AAA = YourClass (99, ' B' );
- AAA = YourClass();
- AAA = YourClass;

## Parameter Passing Methods

- ❖ Efficiency of parameter passing
  - Call-by-value
    - Requires copy be made → Overhead
  - Call-by-reference
    - Placeholder for actual argument
    - Most efficient method
  - Negligible difference for simple types (一般型態沒太大差異)
  - For class types → clear advantage ( Call - by - reference)
- ❖ Call-by-reference desirable
  - Especially for "large" data, like class types

## The const Parameter Modifier

- ❖ Large data types (typically classes)
  - Desirable to use pass-by-reference
  - Even if function will not make modifications
- ❖ Protect argument
  - Use constant parameter
    - Also called constant call-by-reference parameter
  - Place keyword *const* before type
  - Makes parameter "read-only"
  - Attempts to modify result in compiler error

# Example

## ❖ Call by value

```
bool islarge(BankAccount A, BankAccount B)
{
 return (A.getbalance() > B.getbalance());
}
```

## ❖ Call by reference

```
bool islarge(const BankAccount& A, const BankAccount& B)
{
 return (A.getbalance() > B.getbalance());
}
```

# Self-test

```
classYourClass
{
 public :
 YourClass(int newInfo, char moreInfo);
 YourClass();
 void output();
 int myclass;

 private:

 int information;
 char moreInformation;

}

void YourClass::output ()
{
 myclass = 1;
 cout << myclass;
}
```

# Self-test

```
classYourClass
```

```
{
```

```
 public :
```

```
 YourClass(int newInfo, char moreInfo);
```

```
 YourClass();
```

```
 void output() const;
```

```
 int myclass;
```

```
 private:
```

```
 int information;
```

```
 char moreInformation;
```

```
}
```

```
void YourClass::output () const
```

```
{
```

```
 myclass = 1;
```

```
 cout << myclass;
```

```
}
```

```
main()
```

```
{
```

```
 YourClass
```

```
 AAA(43,'A');
```

```
 AAA.output();
```

```
}
```

# Use of const

- ❖ All-or-nothing
- ❖ If no need for function modifications
  - Protect parameter with const
  - Protect ALL such parameters
- ❖ This includes class member function parameters

# Inline Functions

- ❖ For non-member functions:
  - Use keyword *inline* in function declaration and function heading
- ❖ For class member functions:
  - Place implementation (code) for function IN class definition → automatically inline
- ❖ Use for very short functions only
- ❖ Code actually inserted in place of call
  - Eliminates overhead
  - More efficient, but only when short!



widelab  
Web Information and Data Engineering Laboratory

# Inline Member Functions

- ❖ Member function definitions
  - Typically defined separately, in different file
  - Can be defined IN class definition
    - Makes function "in-line"
- ❖ Again: use for very short functions only
- ❖ More efficient
  - If too long → actually less efficient!

# Banks Class Example

```
❖ class Bank
{
public:
 double getbalance() ; }
private:
 int A;
 int B;
};
```

```
❖ double
 Bank::getbalance()
{
 return(A + B*0.01);
```



widelab  
Web Information and Data Engineering Laboratory

# Inline Functions Example

```
class Bank
{
public:
 double getbalance() {return (A+B*0.01);}
private:
 int A;
 int B;
};
```

# Self-test

❖ 請將 `getMonthNumber ()` 和 `getDay()`  
改成 inline function 的 class

# Static Members

- ❖ Static member variables
  - All objects of class "share" one copy
  - One object changes it → all see change
- ❖ Useful for "tracking"
  - How often a member function is called
  - How many objects exist at given time
- ❖ Place keyword *static* before type

# Static Functions

- ❖ Member functions can be static
  - If no access to object data needed
  - And still "must" be member of the class
  - Make it a static function
- ❖ Can then be called outside class
  - From non-class objects:
    - E.g., `Server::getTurn();`
  - As well as via class objects
    - Standard method: `myObject.getTurn();`
- ❖ Can only use static data, functions!

## Static Members Example: Display 7.6 Static Members (1 of 4)

### Display 7.6 Static Members

---

```
1 #include <iostream>
2 using namespace std;

3 class Server
4 {
5 public:
6 Server(char letterName);
7 static int getTurn();
8 void serveOne();
9 static bool stillOpen();
10 private:
11 static int turn;
12 static int lastServed;
13 static bool nowOpen;
14 char name;
15 };

16 int Server::turn = 0;
17 int Server::lastServed = 0;
18 bool Server::nowOpen = true;
```

## Static Members Example: Display 7.6 Static Members (2 of 4)

```
19 int main()
20 {
21 Server s1('A'), s2('B');
22 int number, count;
23 do
24 {
25 cout << "How many in your group? ";
26 cin >> number;
27 cout << "Your turns are: ";
28 for (count = 0; count < number; count++)
29 cout << Server::getTurn() << ' ';
30 cout << endl;
31 s1.serveOne();
32 s2.serveOne();
33 } while (Server::stillOpen());
34
35 cout << "Now closing service.\n";
36
37 return 0;
38 }
```

## Static Members Example: Display 7.6 Static Members (3 of 4)

Display 7.6 Static Members

```
39 Server::Server(char letterName) : name(letterName)
40 { /*Intentionally empty*/}

41 int Server::getTurn()
42 {
43 turn++;
44 return turn;
45 }
46 bool Server::stillOpen()
47 {
48 return nowOpen;
49 }

50 void Server::serveOne()
51 {
52 if (nowOpen && lastServed < turn)
53 {
54 lastServed++;
55 cout << "Server " << name
56 << " now serving " << lastServed << endl;
57 }
```

← Since `getTurn` is static, only static members can be referenced in here.

## Static Members Example: Display 7.6 Static Members (4 of 4)

```
58 if (lastServed >= turn) //Everyone served
59 nowOpen = false;
60 }
```

### SAMPLE DIALOGUE

How many in your group? **3**

Your turns are: 1 2 3

Server A now serving 1

Server B now serving 2

How many in your group? **2**

Your turns are: 4 5

Server A now serving 3

Server B now serving 4

How many in your group? **0**

Your turns are:

Server A now serving 5

Now closing service.

# Summary 1

- ❖ Constructors: automatic initialization of class data
  - Called when objects are declared
  - Constructor has same name as class
- ❖ Default constructor has no parameters
  - Should always be defined
- ❖ Class member variables
  - Can be objects of other classes
    - Require initialization-section

## Summary 2

- ❖ Constant call-by-reference parameters
  - More efficient than call-by-value
- ❖ Can *inline* very short function definitions
  - Can improve efficiency
- ❖ Static member variables
  - Shared by all objects of a class